

Django Interview Questions & Answers

Here are comprehensive, interview-ready answers organized by topic.

Django Basics

1. What is Django?

Django is a high-level Python web framework that encourages rapid development and clean, pragmatic design. It follows the "batteries-included" philosophy, providing almost everything developers need to build web applications out of the box.

2. Why is Django called a batteries-included framework?

Because it comes with a wide range of built-in features right out of the box, including:

- ORM (Object-Relational Mapper)
- Authentication system
- Admin panel
- URL routing
- Form handling and validation
- Session management
- Caching
- CSRF protection

You don't need to assemble third-party tools—everything works seamlessly together with consistent design principles.

3. What are the features of Django?

- ORM – write Python instead of SQL to interact with the database
- URL routing – clean, regex-based or path-based URL patterns
- Template engine – Pythonic HTML templating with inheritance
- Forms – automatic form generation and validation

- Authentication – built-in user management
- Admin interface – auto-generated admin panel
- Migrations – schema version control
- Security – CSRF, XSS, SQL injection protection
- Internationalization – multi-language support
- Caching framework – multiple cache backends

4. What are the advantages of Django?

- Rapid development – pre-built modules speed up common tasks
- Secure – protects against common web vulnerabilities
- Scalable – used by Instagram, Spotify, and other large platforms
- Versatile – can build anything from CMS to social networks
- Well-documented – extensive and up-to-date documentation
- Large community – abundant third-party packages and support

5. What is the difference between Django and Flask?

Django	Flask
"Batteries-included" – full-stack framework	Micro-framework – minimal core
Built-in ORM, admin, auth, forms	You choose your own components
More opinionated (Django way)	Flexible, unopinionated
Heavier, more features	Lightweight, minimal
Steeper learning curve	Easier to get started
Better for large, complex applications	Better for small APIs or microservices

6. Explain the MTV architecture.

Django follows a modified MVC pattern called Model-Template-View (MTV):

- Model – defines data structure and handles database interactions
- Template – presentation layer (HTML with Django template tags) – how data is presented
- View – business logic that processes requests, interacts with models, and renders templates

The framework itself acts as the controller, mapping URLs to views and handling the request-response cycle.

7. What is the role of `manage.py`?

A command-line utility that lets you interact with your Django project. Common commands:

- `python manage.py runserver` – start development server
- `python manage.py makemigrations` – create migration files
- `python manage.py migrate` – apply migrations
- `python manage.py createsuperuser` – create admin user
- `python manage.py shell` – open Django shell

8. What is the difference between a Django project and a Django app?

Project	App
A container for apps	A self-contained module with models, views, and code
Contains settings, not database models	Contains models, views, templates, URLs, etc.
One project per website	Many apps can exist within a project

Cannot be reused across projects

Can be reused across multiple projects

9. How do you create a Django project?

```
bash
```

```
django-admin startproject myproject
```

10. How do you create a Django app?

```
bash
```

```
python manage.py startapp myapp
```

Project Structure

11. What is the purpose of `settings.py`?

Contains all configuration for the Django project:

- Database configuration
- Installed apps list
- Middleware
- Templates settings
- Static/media file settings
- Security settings (SECRET_KEY, DEBUG, ALLOWED_HOSTS)
- Internationalization settings

12. What is the purpose of `urls.py`?

Defines URL patterns and maps them to views. It's the routing table for your application – when a request comes in, Django matches the URL against these patterns and calls the corresponding view.

13. What is the purpose of `views.py`?

Contains the business logic – functions or classes that process HTTP requests, interact with models, and return HTTP responses (usually rendered templates or JSON).

14. What is the purpose of `models.py`?

Defines the data structure for your application. Each model is a Python class that maps to a database table. It defines fields, relationships, and methods for data manipulation.

15. What is the purpose of `admin.py`?

Registers models with Django's built-in admin interface, allowing you to manage data through a web-based dashboard. You can also customize how models are displayed and filtered.

16. What is the purpose of `apps.py`?

Contains configuration for the app – defines the app name, default settings, and provides a `ready()` method for startup code (signals, etc.).

17. What is the purpose of `manage.py`?

(Answered in Q7)

Models & Database

18. What is a Django model?

A Python class that defines the structure of your database table. Each model class maps to a single database table, and each attribute maps to a database column.

19. What is ORM?

Object-Relational Mapper – a technique that lets you interact with your database using Python objects instead of writing raw SQL. Django's ORM automatically translates Python code to SQL queries.

20. What are migrations?

Migrations are Python files that track changes to your models and translate them into database schema changes. They allow you to evolve your database schema over time without losing data.

21. What is the difference between `makemigrations` and `migrate`?

`makemigrations`

`migrate`

Creates migration files based on changes to your models

Applies the migrations to the database

Doesn't modify the database

Actually executes the SQL and changes the schema

Generates Python files in the `migrations/` folder

Reads the migration files and runs them

22. What is the purpose of the `__str__()` method?

Defines how a model instance is displayed as a string (e.g., in the admin panel, or when printed). It improves readability and debugging.

23. How do you create a model?

```
python

from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    age = models.IntegerField()
    email = models.EmailField(unique=True)

    def __str__(self):

        return self.name
```

24. What field types are available in Django models?

- CharField – short strings
- TextField – long text
- IntegerField – integers
- BooleanField – True/False
- DateField / DateTimeField – dates and times
- EmailField – email validation
- URLField – URL validation
- ForeignKey – one-to-many relationships
- ManyToManyField – many-to-many relationships
- OneToOneField – one-to-one relationships
- DecimalField / FloatField – decimal numbers
- FileField / ImageField – file uploads

25. What is a primary key?

A unique identifier for each record in a database table. Django automatically adds an `id` field as the primary key if you don't specify one. You can use `primary_key=True` on any field to override this.

26. What is the `Meta` class in models?

An inner class that provides metadata about the model:

- `db_table` – custom table name
 - `ordering` – default ordering for queries
 - `unique_together` – composite uniqueness constraints
 - `verbose_name` / `verbose_name_plural` – human-readable names
 - `indexes` – custom database indexes
-

ORM Queries

27. What is a `QuerySet`?

A `QuerySet` is a collection of database query results. It's lazy – the database query is not executed until the `QuerySet` is evaluated (e.g., iterated, sliced, or converted to a list).

28. What is the difference between `get()` and `filter()`?

`get()`

`filter()`

Returns a single model instance

Returns a `QuerySet` (can contain 0, 1, or many objects)

Raises `DoesNotExist` if none found

Returns empty `QuerySet` if none found

Raises `MultipleObjectsReturned` if multiple found

Returns all matches (no error)

Use when you expect exactly one result

Use when you may have zero or multiple results

29. What is the difference between `all()` and `values()`?

`all()`

`values()`

Returns a `QuerySet` of model instances

Returns a `QuerySet` of dictionaries

Access fields via attributes: `obj.name`

Access fields via keys: `dict['name']`

Lazy evaluation

Lazy evaluation

Can access related objects and methods

Only returns specified fields (more efficient)

30. How do you insert data using ORM?

python

```
# Method 1: create()
```

```
student = Student.objects.create(name="Alice", age=20,  
email="alice@example.com")
```

```
# Method 2: save()
```

```
student = Student(name="Bob", age=22)
```

```
student.save()
```

31. How do you update records?

python

Single record update

```
student = Student.objects.get(id=1)
student.age = 21
student.save()
```

Bulk update

```
Student.objects.filter(age__lt=18).update(status="minor")
```

32. How do you delete records?

python

Single record

```
student = Student.objects.get(id=1)
student.delete()
```

Bulk delete

```
Student.objects.filter(age__lt=18).delete()
```

33. How do you count records?

python

```
count = Student.objects.count()           # All records
```

```
count = Student.objects.filter(age=20).count() # Filtered
```

34. How do you sort records?

python

Ascending

```
students = Student.objects.order_by('name')
```

```
# Descending
students = Student.objects.order_by('-age')

# Multiple fields
students = Student.objects.order_by('age', '-name')
```

35. What are ORM lookups?

Field lookups are used with `filter()`, `exclude()`, and `get()` to specify conditions. They use a double-underscore syntax: `field__lookuptype=value`

36. Explain `contains`, `icontains`, `startswith`, `endswith`, `gt`, `lt`, `in`, and `range`.

Lookup	Description	Example
<code>contains</code>	Case-sensitive containment	<code>name__contains='abc'</code>
<code>icontains</code>	Case-insensitive containment	<code>name__icontains='abc'</code>
<code>startswith</code>	Case-sensitive prefix	<code>name__startswith='A'</code>
<code>endswith</code>	Case-sensitive suffix	<code>name__endswith='z'</code>
<code>gt</code>	Greater than	<code>age__gt=18</code>
<code>lt</code>	Less than	<code>age__lt=18</code>
<code>gte</code> / <code>lte</code>	Greater/less than or equal	<code>age__gte=18</code>

<code>in</code>	In a list	<code>age__in=[18, 20, 22]</code>
<code>range</code>	Between two values	<code>age__range=(18, 25)</code>
<code>exact</code>	Exact match (default)	<code>name__exact='Alice'</code>
<code>iexact</code>	Case-insensitive exact	<code>name__iexact='alice'</code>

Views

37. What is a view?

A view is a Python function or class that receives an HTTP request and returns an HTTP response. It contains the business logic for your application – processing data, interacting with models, and rendering templates.

38. What is the difference between Function-Based Views (FBVs) and Class-Based Views (CBVs)?

Function-Based Views (FBVs)	Class-Based Views (CBVs)
Simple, explicit, easy to understand	More structured, reusable
Use conditional branching for HTTP methods	Use separate methods for each HTTP method
Less code for simple views	More code upfront, but less repetition for common patterns

Harder to extend	Easy to extend via inheritance
Better for beginners	Better for complex, repetitive patterns

39. What are HTTP methods?

The standard HTTP verbs used to indicate the desired action on a resource:

- GET – retrieve data
- POST – submit/create data
- PUT – update/replace a resource
- PATCH – partial update
- DELETE – delete a resource

40. What is the difference between GET and POST?

GET	POST
Appends data to URL (query string)	Sends data in request body
Visible in browser history	Not visible in URL
Limited data size	No size limit
Can be cached and bookmarked	Not cached or bookmarked
Should be idempotent (no side effects)	Changes server state
Use for search, filtering, reading	Use for forms, login, data submission

41. How do you pass data from a view to a template?

Using the context dictionary in the `render()` function:

python

```
def student_list(request):
    students = Student.objects.all()
    context = {
        'students': students,
        'title': 'Student Directory',
        'count': students.count()
    }

    return render(request, 'students/list.html', context)
```

42. What is `render()`?

A shortcut function that:

1. Loads a template
2. Renders it with the provided context
3. Returns an `HttpResponse` object with the rendered content

python

```
from django.shortcuts import render

return render(request, 'template.html', context)
```

43. What is `redirect()`?

A shortcut that returns an `HttpResponseRedirect` to a specified URL:

python

```
from django.shortcuts import redirect

return redirect('home')          # by URL name
return redirect('/dashboard/')    # by path

return redirect('student_detail', student_id=1) # with parameters
```

URL Routing

44. What is URL routing?

The process of mapping incoming URLs to the appropriate views. Django matches the URL path against defined patterns and calls the corresponding view function or class.

45. What is the `path()` function?

Used in `urls.py` to define URL patterns:

```
python

from django.urls import path
from . import views

urlpatterns = [
    path('students/', views.student_list, name='student_list'),
    path('students/<int:id>/', views.student_detail, name='student_detail'),
]
```

46. What is `include()`?

Used to include other URL configurations – keeps your project organized by delegating URL routing to individual apps:

```
python

from django.urls import include, path

urlpatterns = [
    path('students/', include('students.urls')),
    path('teachers/', include('teachers.urls')),
]
```

47. What are URL parameters?

Dynamic parts of a URL that capture values and pass them to views:

- `<int:id>` – captures an integer
- `<str:name>` – captures a string
- `<slug:slug>` – captures a slug
- `<uuid:uuid>` – captures a UUID
- `<path:path>` – captures a full path

48. How do you reverse URLs?

Using the `reverse()` function or the `{% url %}` template tag:

```
python
```

```
# In views
```

```
from django.urls import reverse
```

```
url = reverse('student_detail', args=[1]) # '/students/1/'
```

```
# In templates
```

```
<a href="{% url 'student_detail' student.id %}">View</a>
```

Templates

49. What is the Django template engine?

Django's built-in templating system that allows you to generate HTML dynamically. It uses its own language with template tags (`{% %}`) and variables (`{{ }}`).

50. What are template tags?

Special syntax (`{% ... %}`) that performs logic in templates:

- `{% for %}` – loops
- `{% if %}` – conditionals
- `{% block %}` – defines a block for inheritance
- `{% extends %}` – inherits from a parent template
- `{% include %}` – includes another template
- `{% url %}` – generates URLs
- `{% load %}` – loads custom template tags

51. What are template filters?

Modify variables for display using the pipe (`|`) syntax:

- `{{ name|upper }}` – uppercase
- `{{ date|date:"Y-m-d" }}` – format date
- `{{ text|truncatewords:30 }}` – truncate
- `{{ value|default:"N/A" }}` – default value
- `{{ list|join:", " }}` – join list

52. How do you write loops in templates?

django

```
{% for student in students %}
    <li>{{ student.name }} - {{ student.age }}</li>
{% empty %}
    <li>No students found</li>

{% endfor %}
```

53. How do you write conditions in templates?

django

```
{% if student.age >= 18 %}
    <p>Adult</p>
{% elif student.age >= 13 %}
    <p>Teenager</p>
{% else %}
```

```
<p>Child</p>

{% endif %}
```

54. What is template inheritance?

A powerful feature that allows you to create a base template with common structure (header, footer, navigation) and extend it in child templates, overriding specific blocks.

55. What are `{% extends %}` and `{% block %}`?

- `{% extends "base.html" %}` – tells Django to use `base.html` as the parent template
- `{% block content %}` – defines a region that child templates can override

base.html:

```
django

<html>
<head><title>{% block title %}My Site{% endblock %}</title></head>
<body>
    {% block content %}{% endblock %}
</body>

</html>
```

child.html:

```
django

{% extends "base.html" %}
{% block title %}Student List{% endblock %}
{% block content %}
    <h1>Students</h1>
    <!-- content here -->

{% endblock %}
```

Forms

56. What is a Django Form?

A Python class that handles form rendering, validation, and data processing. It reduces boilerplate and provides built-in security.

57. What is a ModelForm?

A specialized form that automatically generates fields from a Django model:

```
python

from django.forms import ModelForm
from .models import Student

class StudentForm(ModelForm):
    class Meta:
        model = Student

        fields = ['name', 'age', 'email']
```

58. What is form validation?

The process of ensuring submitted data meets the required criteria. Django provides:

- Field-level validation – via `clean_<fieldname>()` methods
- Form-level validation – via `clean()` method
- Built-in validators – on field definitions

59. How do you display validation errors?

```
django

{% if form.errors %}
    <div class="errors">
        {% for field, errors in form.errors.items %}
```

```

        {% for error in errors %}
            <p>{{ error }}</p>
        {% endfor %}
    {% endfor %}
</div>
{% endif %}

{% for field in form %}
    {{ field.label_tag }}
    {{ field }}
    {{ field.errors }}

{% endfor %}

```

60. What is CSRF protection?

Cross-Site Request Forgery protection prevents malicious websites from submitting unauthorized requests. Django includes CSRF tokens in forms:

```

django

<form method="post">
    {% csrf_token %}
    {{ form }}
    <button type="submit">Submit</button>

</form>

```

Admin Panel

61. What is the Django Admin Panel?

An auto-generated, feature-rich administrative interface for managing your application's data. It reads your models and provides a CRUD (Create, Read, Update, Delete) interface.

62. How do you register a model in the admin?

In `admin.py`:

```
python
```

```
from django.contrib import admin
from .models import Student
```

```
admin.site.register(Student)
```

63. How do you create a superuser?

```
bash
```

```
python manage.py createsuperuser
```

64. How do you customize the admin panel?

```
python
```

```
@admin.register(Student)
class StudentAdmin(admin.ModelAdmin):
    list_display = ['name', 'age', 'email', 'created_at']
    list_filter = ['age', 'created_at']
    search_fields = ['name', 'email']
    ordering = ['-created_at']
    fieldsets = (
        ('Personal Info', {'fields': ('name', 'age', 'email')}),
        ('Timestamps', {'fields': ('created_at', 'updated_at')}),
    )
```

Authentication

65. What is authentication?

The process of verifying identity – confirming that a user is who they claim to be (login).

66. What is authorization?

The process of determining permissions – what an authenticated user is allowed to do.

67. How do you create a user?

```
python

from django.contrib.auth.models import User

# Create user with password (hashed automatically)

user = User.objects.create_user('username', 'email@example.com',
'password123')
```

68. How do you log in a user?

```
python

from django.contrib.auth import authenticate, login

def login_view(request):
    user = authenticate(request, username='alice', password='secret')
    if user is not None:
        login(request, user)

        return redirect('home')
```

69. How do you log out a user?

```
python

from django.contrib.auth import logout

def logout_view(request):
    logout(request)
```

```
return redirect('home')
```

70. What is `login_required`?

A decorator that restricts access to views – only authenticated users can access them:

python

```
from django.contrib.auth.decorators import login_required
```

```
@login_required
```

```
def profile_view(request):
```

```
    return render(request, 'profile.html')
```

Static & Media Files

71. What are static files?

Assets that don't change and are not user-uploaded: CSS, JavaScript, images, fonts.

72. What are media files?

User-uploaded content: profile pictures, documents, videos.

73. What is the difference between static and media files?

Static Files	Media Files
Part of the application codebase	User-generated content

Same for all users	Unique per user
Collected with <code>collectstatic</code>	Stored in <code>MEDIA_ROOT</code>
Usually in version control	Not in version control

74. What are `STATIC_URL` and `MEDIA_URL`?

URL prefixes used to serve files:

- `STATIC_URL = '/static/'` – the URL for static files
- `MEDIA_URL = '/media/'` – the URL for media files

75. What are `STATIC_ROOT` and `MEDIA_ROOT`?

Filesystem paths:

- `STATIC_ROOT` – directory where `collectstatic` collects all static files for production
 - `MEDIA_ROOT` – absolute filesystem path where user-uploaded files are stored
-

Relationships

76. What is `ForeignKey`?

A one-to-many relationship – one model has many instances of another:

python

```
class Author(models.Model):  
    name = models.CharField(max_length=100)
```

```
class Book(models.Model):
    title = models.CharField(max_length=200)

    author = models.ForeignKey(Author, on_delete=models.CASCADE)
```

77. What is OneToOneField?

A one-to-one relationship – each record in one model relates to exactly one record in another:

python

```
class UserProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE)

    bio = models.TextField()
```

78. What is ManyToManyField?

A many-to-many relationship – records from one model can relate to many records in another, and vice versa:

python

```
class Student(models.Model):
    name = models.CharField(max_length=100)

class Course(models.Model):
    name = models.CharField(max_length=100)

    students = models.ManyToManyField(Student)
```

79. Explain model relationships with examples.

Relationship

Example

Django Field

One-to-Many	Author → Books	ForeignKey
One-to-One	User → Profile	OneToOneField
Many-to-Many	Students ↔ Courses	ManyToManyField

Middleware

80. What is middleware?

Middleware is a framework of hooks into Django's request/response processing. It's a lightweight plugin that processes each request and response globally.

81. Name some built-in middleware.

- `AuthenticationMiddleware` – associates users with requests
- `SessionMiddleware` – enables session support
- `CsrfViewMiddleware` – CSRF protection
- `SecurityMiddleware` – security headers
- `CommonMiddleware` – URL rewriting, etags
- `MessageMiddleware` – message framework
- `XFrameOptionsMiddleware` – clickjacking protection

82. How does middleware work?

Middleware is processed in a chain:

1. Request phase – middleware processes the request in the order they're listed (top to bottom)
2. View phase – the view is called
3. Response phase – middleware processes the response in reverse order (bottom to top)

Each middleware can:

- Process the request before it reaches the view
 - Process the response before it's sent to the client
 - Short-circuit the request/response cycle
-

Sessions & Cookies

83. What is a session?

A server-side mechanism to store user-specific data across multiple requests. Sessions are identified by a session ID stored in a cookie.

84. What is a cookie?

A small piece of data stored on the client's browser. Cookies are sent with every request to the same domain.

85. What is the difference between sessions and cookies?

Sessions	Cookies
Stored server-side	Stored client-side (browser)
More secure (data not exposed)	Less secure (data visible to user)
Can store large amounts of data	Limited size (~4KB)
Data persists until session expires	Can be set to expire at a specific time

Uses a session ID cookie for identification

Stores data directly

Security

86. What is CSRF?

Cross-Site Request Forgery – an attack where a malicious website tricks a user into making an unwanted request to a website where they're authenticated. Django prevents this with CSRF tokens.

87. What is XSS?

Cross-Site Scripting – an attack where malicious scripts are injected into web pages viewed by other users. Django's template engine automatically escapes output to prevent XSS.

88. What is SQL Injection?

An attack where malicious SQL queries are injected through form inputs. Django's ORM uses parameterized queries, which makes SQL injection impossible when using the ORM correctly.

89. How does Django prevent SQL Injection?

- ORM uses parameterized queries – user input is treated as data, not executable code
 - Raw SQL should use `cursor.execute(sql, params)` with parameter binding
 - Never use string concatenation for SQL queries
-

Django REST Framework (Basic)

90. What is Django REST Framework?

A powerful and flexible toolkit for building Web APIs in Django. It provides:

- Serializers for data conversion
- Browsable API
- Authentication and permissions
- ViewSets and Routers

91. What is a serializer?

Serializers convert complex data (QuerySets, model instances) to Python datatypes that can be rendered into JSON, XML, or other formats. They also handle deserialization (validating and saving incoming data).

92. What is JSON?

JavaScript Object Notation – a lightweight data-interchange format that is human-readable and easy for machines to parse. It's the most common format for REST APIs.

93. What is an API?

Application Programming Interface – a set of rules and protocols that allows different software applications to communicate with each other.

94. What are RESTful APIs?

APIs that follow REST (Representational State Transfer) principles:

- Stateless
- Resource-based (using URLs)
- HTTP methods (GET, POST, PUT, DELETE)
- JSON/XML responses
- HATEOAS (hypermedia)

95. What is the difference between PUT and PATCH?

PUT	PATCH
Replaces the entire resource	Partially updates a resource
Sends the complete resource data	Sends only the fields to update
Idempotent (same request → same result)	Not necessarily idempotent
If a field is missing, it's set to default/null	Missing fields are left unchanged

Deployment

96. What is WSGI?

Web Server Gateway Interface – a standard interface between web servers and Python web applications. Django uses WSGI for synchronous (traditional) deployment.

97. What is ASGI?

Asynchronous Server Gateway Interface – the successor to WSGI that supports asynchronous operations. Django supports ASGI for WebSockets, long-polling, and other async features.

98. What is the default database in Django?

SQLite (lightweight, file-based database).

99. Which databases does Django support?

- Default: SQLite
- Official support: PostgreSQL, MySQL, Oracle
- Third-party: MongoDB (via Django), CockroachDB, and others

100. How do you deploy a Django application?

Common approaches:

1. Traditional hosting – Apache/nginx + WSGI (Gunicorn, uWSGI)
2. Platform as a Service – Heroku, PythonAnywhere, AWS Elastic Beanstalk
3. Containerized – Docker + Kubernetes
4. Serverless – AWS Lambda (with Zappa)

Steps typically include:

- Set `DEBUG = False`
 - Configure `ALLOWED_HOSTS`
 - Run `collectstatic`
 - Set up a production database
 - Use environment variables for secrets
 - Configure a reverse proxy (nginx) and WSGI server (Gunicorn)
-

Git & Project-Based Questions

101. Explain your Django project.

Structure your answer:

- Problem – what was the project about?
- Solution – how did Django help?
- Key features – what did you build?
- Technical choices – why Django? Why that database?
- Challenges – what was difficult and how did you solve it?
- Outcome – what did you learn?

102. Which database did you use and why?

- SQLite – for development (lightweight, no setup)
- PostgreSQL – for production (robust, supports advanced features like JSON, full-text search)
- MySQL – if the organization already uses it

103. What challenges did you face during development?

Common challenges and how to present them:

- Performance issues – optimized queries with `select_related()` and `prefetch_related()`
- Authentication – implemented JWT or session-based auth
- File uploads – handled large files with chunked uploads
- Deployment – configured environment variables and static files

104. How did you use Git in your project?

- Feature branches: `feature/login`, `feature/dashboard`
- Commit messages: descriptive, following conventional commits
- Pull requests: code reviews before merging
- Tagging: version tags (`v1.0.0`)

105. How do you debug Django applications?

- `print()` / `pprint()` for quick checks
- `import pdb; pdb.set_trace()` for interactive debugging
- Django Debug Toolbar – shows SQL queries, performance metrics
- `logging` module for production logging
- `django-extensions` – `shell_plus` for better shell

106. Have you worked with REST APIs?

If yes: Mention DRF, serializers, authentication, API versioning, documentation (Swagger/DRF-YASG).

If no: Mention understanding of REST concepts and willingness to learn.

107. What is the request-response lifecycle in Django?

1. Browser sends HTTP request
2. Web server (nginx/Apache) receives it
3. WSGI/ASGI server (Gunicorn/Uvicorn) passes it to Django
4. Middleware processes the request
5. URL resolver finds the matching view
6. View executes business logic (may query the database via ORM)
7. View renders a template or returns JSON
8. Response goes back through middleware (in reverse)
9. Web server sends the response to the browser

108. How do you optimize Django queries?

- Use `select_related()` for foreign key relationships (JOINS)
- Use `prefetch_related()` for many-to-many and reverse foreign keys
- Use `only()` and `defer()` to select only needed fields
- Use `values()` or `values_list()` for specific fields
- Add database indexes with `db_index=True` or `Meta.indexes`
- Use `count()` instead of `len(queryset)`
- Use `exists()` to check existence without fetching objects

109. What are the common Django commands you use?

Command	Purpose
<code>runserver</code>	Start development server
<code>makemigrations</code>	Create migration files

<code>migrate</code>	Apply migrations
<code>createsuperuser</code>	Create admin user
<code>shell</code>	Open Django interactive shell
<code>collectstatic</code>	Collect static files
<code>test</code>	Run tests
<code>startapp</code>	Create a new app
<code>startproject</code>	Create a new project
<code>dumpdata</code> / <code>loaddata</code>	Export/import data

110. Why should we hire you as a Django developer?

Structure your answer:

- Technical skills – Python, Django, ORM, REST APIs, database design
- Problem-solving – experience debugging and optimizing
- Learning mindset – stay updated with Django versions and best practices
- Team player – Git, code reviews, collaboration
- Results – show projects you've built
- Passion – genuine interest in web development and Django